

Lekcija 1 – Uvod

Pregled

- 1.1 Motivacija**
- 1.2 Osnovni pojmovi**
- 1.3 Temeljni principi**

Ciljevi lekcije

- U ovoj lekciji:
 - Upoznavanje sa istorijatom struktiranog i OO programiranja.
 - Upoznavanje sa osnovnim pojmovima i principima vezanim za OO programiranje
 - Pregled koncepata koji će biti izučavani u ostatku kursa

1.1 Motivacija

- Stalan rast složenosti u poslovanju i napredak računarske tehnike ima za posljedicu porast pritiska na softversku industriju
 - Kada naraslim zahtjevima nije moguće odgovoriti dolazi do tzv. “softverske krize”
 - Krize se prevazilaze uvođenjem novih alata i tehnika
 - Ovaj proces je cikličan
 - Razvoj softvera prati ciklično smjenjivanje povećanja pritiska na softversku industriju usljed rasta složenosti poslovanja i smanjena pritisaka usljed uvođenja novih alata i tehnika (D.S. Frankel)

1.1 Motivacija

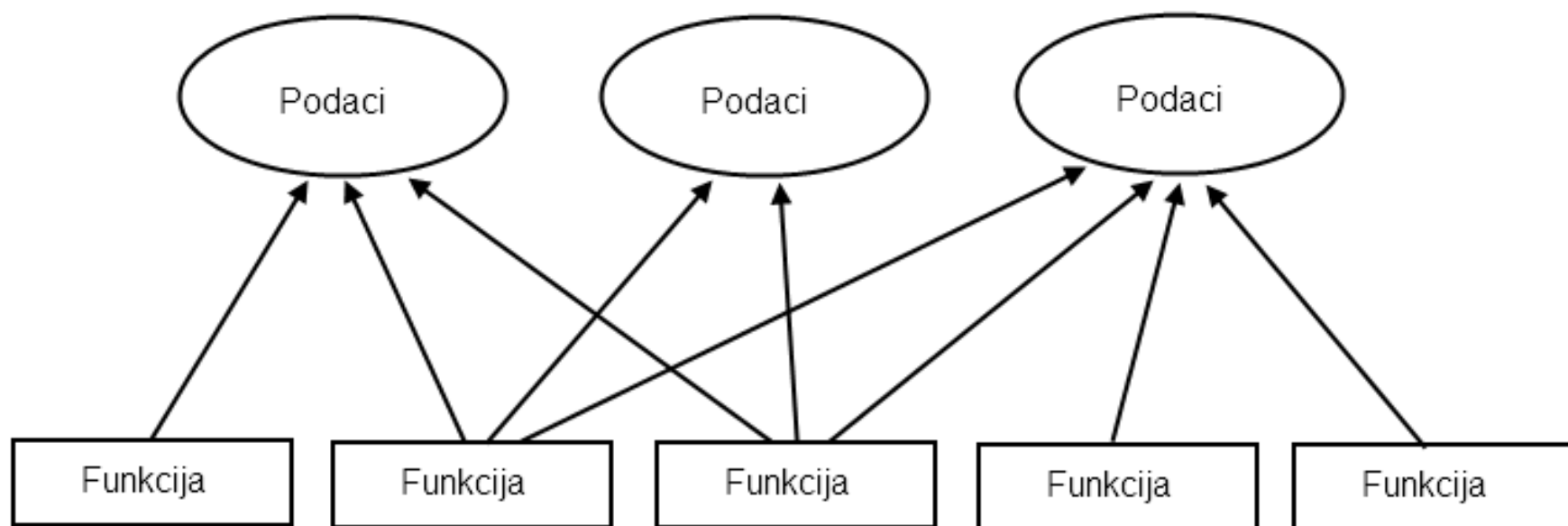
- Jedna takva kriza javila se šezdesetih godina
 - U to vrijeme nije postojala tehnologija razvoja programa, pa se za takav pristup koristi naziv “haotično programiranje”
 - U programima se često koristila naredba skoka (*goto*) pa se takav programski kod nazivao “špageti kod”
 - Strukturirano programiranje nastaje kao odgovor na narasle zahtjeve
 - Uvodi se red u programiranje upotrebom strogo definisanih upravljačkih struktura (*if*, *for* itd.)
 - Povezano je sa imperativnom paradigmom programiranja

1.1 Motivacija

- Strukturirano programiranje
 - Ogroman napredak u odnosu na programiranje putem asemblera
 - Program predstavlja niz naredbi
 - Kao što je rečeno koriste se propisane naredbe
 - Veći programi koji imaju nekoliko stotina instrukcija postaju nerazumljivi
 - Vrš se razbijanje na manje procedure i funkcije
 - Čvrsto je vezano sa metodom sukcesivne dekompozicija
 - Problem se postepeno dekomponuje i raščlanjuje na manje jedinice (procedure i funkcije)

1.1 Motivacija

- Pored brojnih prednosti i postoje i mane
 - Program je podijeljen na dvije velike cjeline, podaci i kod
 - Za složene programe ovo može biti problem
 - Nedovoljno razvijena podrška za uvođenje novih tipova podataka i proširenje postojećih
 - Nedovoljno razvijeni mehanizmi za povezivanje modula koda napisanih od strane različitih timova programera
 - Nedostatak mehanizama za ponovnu upotrebu koda
- Kako složenost softvera stalno raste navedeni nedostaci predstavljaju ozbiljno ograničenje u upotrebi strukturiranog programiranja
 - OO programiranje je dijelom uvedeno kako bi se prevazišli nedostaci tradicionalnog programiranja
 - OO programiranje ne znači kompletan nestanak strukturiranog programiranja, već su principi strukturiranog programiranja prisutni u OO programskim jezicima



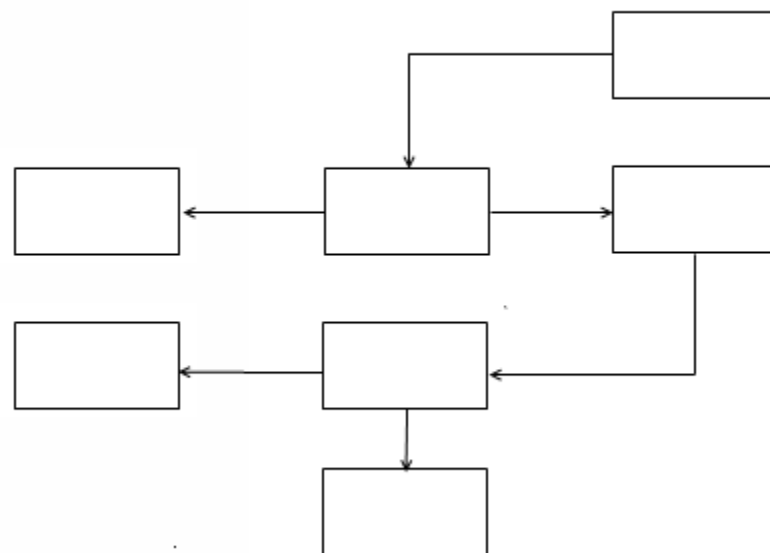
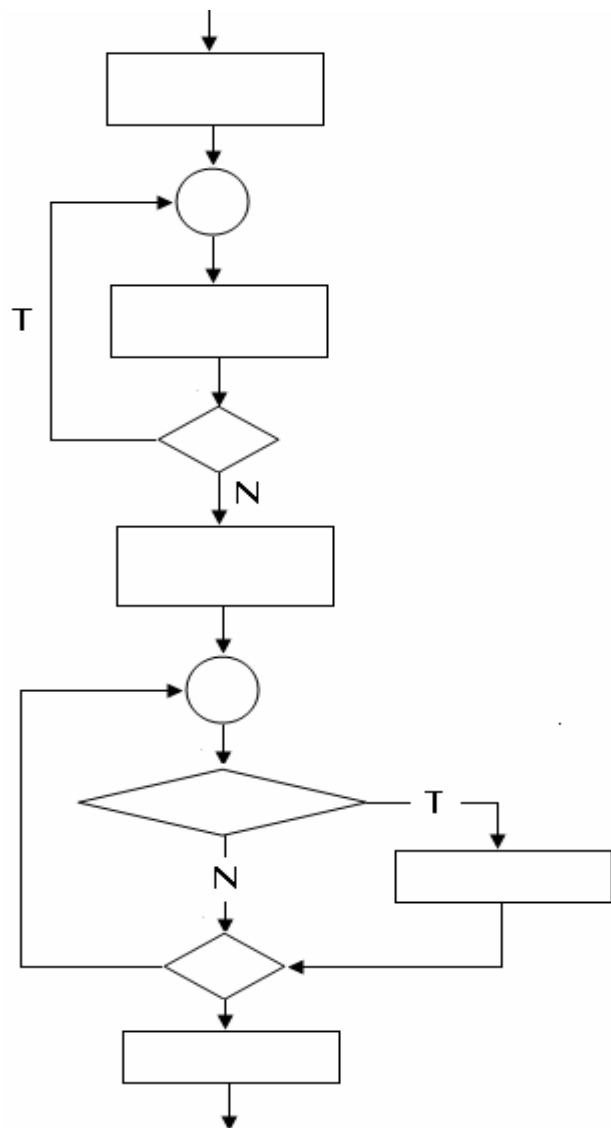
Slika 1. Proceduralna paradigma

1.2 Osnovni pojmovi

- Za prevazilaženje poteškoća uzrokovanih složenošću često se pribjegava upotrebi modela
 - Mapa je primjer modela, kao i maketa zgrade
- Modeli se sastoje od elemenata koji opisuju neku fizičku, apstraktnu ili hipotetičku realnost
 - Model je pojednostavljenja slika svijeta
 - Ta “slika” sadrži samo one detalje koji su od interesa
- Upotreba modela je široka i prisutna u mnogim domenima
- Ima očigledne prednosti:
 - Lakše je sprovoditi eksperimente i analizu nad modelima, jer su jednostavniji i njihova izrada košta mnogo manje nego izrada samog sistema
 - Povećava se nivo apstrakcije
 - Lakša komunikacija između članova razvojnog tima.

1.2 Osnovni pojmovi

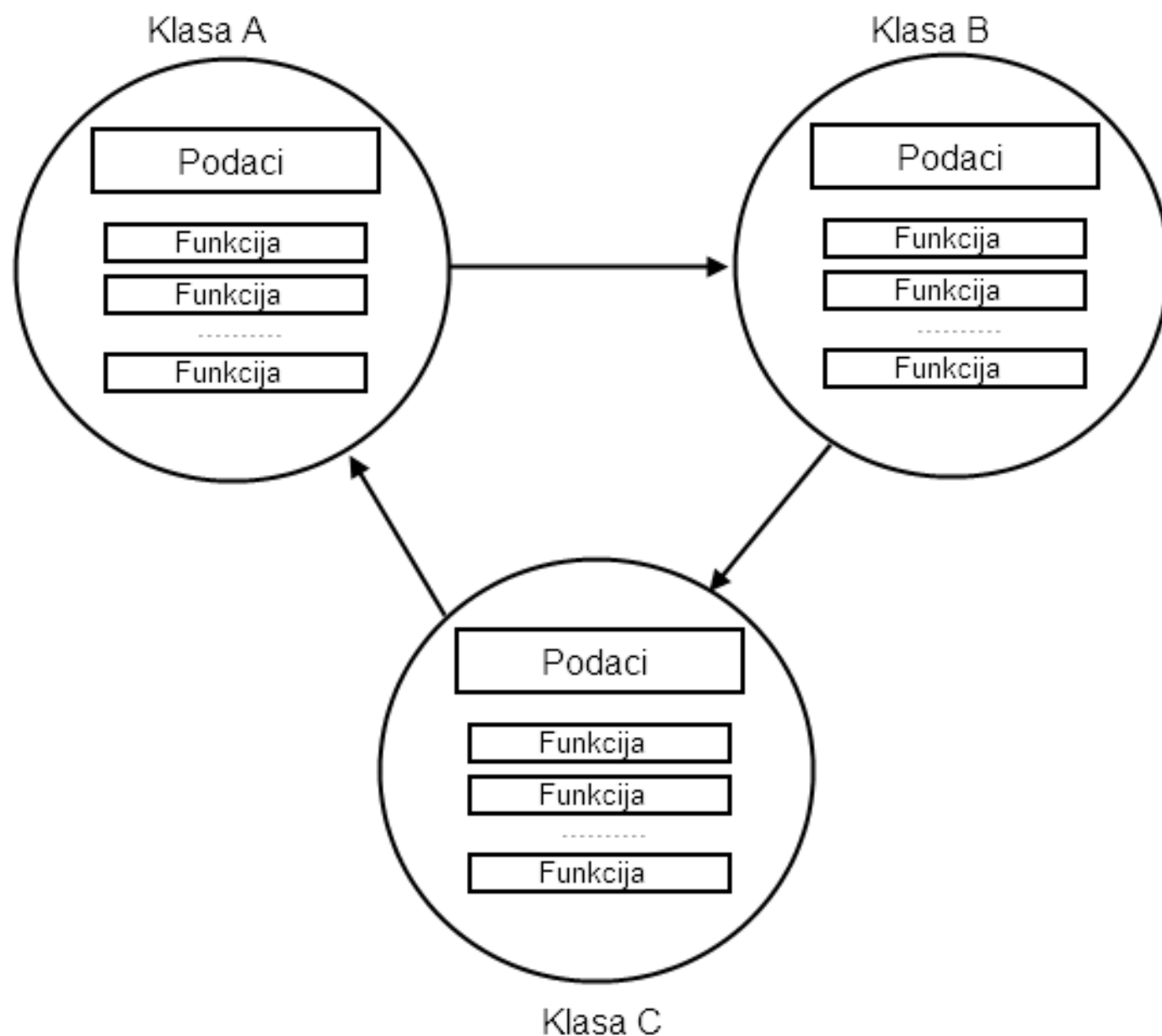
- U OO pristupu uvodi se bitno drugačiji način razmišljanja u programiranje
- U OO pristupu softver se tretira kao model realnog svijeta
- Svijet se sastoji iz entiteta (objekata)
 - Entiteti imaju svoja stanja (atribute) i ponašanja (akcije, procedure)
 - Koncepti kao što su klase, metode i nasljeđivanje služe za modelovanje (opisivanje) entiteta
 - Klasa je glavni koncept koji predstavlja apstrakciju (model) jednog entiteta



Slika. 2. Imperativno vs OO

1.2 Osnovni pojmovi

- Princip je da se podaci i programski kod vezani za jedan entitet čuvaju u okviru jedne programske jedinice (klasa)
 - Najčešće, ovaj princip realizuje su putem koncepta klase
 - Postiže se bolja sprega programskog koda sa podacima
 - Lakše je kontrolisati više manjih jedinica
 - Znatno unapređenje u odnosu na strukturirano programiranje gdje postoje dvije velike odvojene cjeline, tj. programski kod i podaci
 - Podaci vezani za jedan entitet dostupni su samo funkcijama i procedurama vezanim za taj entitet
 - Pojednostavljuje se traženje grešaka
 - Analogija sa preduzećem
 - Sastoji se iz više odvojenih departmana i svako ima svoja zaduženja
 - Npr. računovodstvo vodi računa o zaradama i samo su njima dostupni ti podaci
 - Važi princip “crne kutije”
 - Ne ulazi se u detalje implementacije klase koja modelira neki entitet
 - Sakrivanje složenosti



Slika 3. OO paradigma

1.2 Osnovni pojmovi

- OO pristup stvara osnove za modelovanje rješenja
 - Uočavaju se entiteti, njihova stanja i akcije
 - Postoje jezici za modelovanje putem kojih je ove opise moguće i formalno zapisati i sačuvati (UML)
 - Rješenje se potom implementira u nekom konkretnom OO programskom jeziku koristeći koncepte OO programiranja
- Treba razlikovati OO modelovanje (projektovanje) i OO programiranje
 - OO modelovanje je pristup koji se bavi kreiranjem modela realnog svijeta
 - OO programiranje je tehnika za realizaciju kreiranog modela

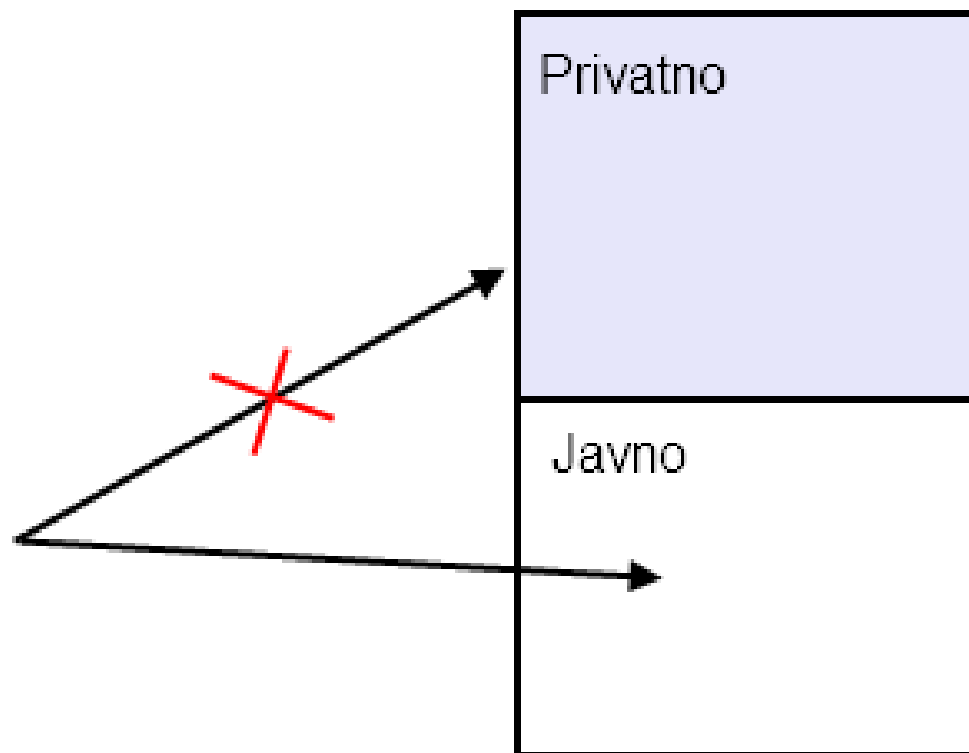
1.3 Temeljni principi – Apstraktni tipovi

- Apstraktni tipovi podatka
 - Apstrakcija:
 - Prepoznavanje bitnih svojstava skupova sličnih objekata
 - Opisivanje novim tipom podatka
 - Pored ugrađenih tipova podataka, u OO jezicima moguće je definisati i sopstvene tipove podataka
 - Oni se ravnopravno koriste sa ugrađenim tipovima
 - Moguće je kreirati proizvoljan broj promjenjivih za taj novi tip
 - Instanciranje
 - Moguće je definisati i operacije nad novim tipom
 - Novi tip se uglavnom realizuje putem klasa
 - Novi tipovi podataka odgovaraju pojmovima iz realnog svijeta, tj. domena problema
 - Time se postiže bolja razumljivost programskog koda i olakšava programiranje
 - Npr. novi tipovi mogu da predstavljaju studente, zgrade, geometrijske objekte itd.

1.3 Temeljni principi – Enkapsulacija

- Enkapsulacija

- Implementacija nekog tipa može (treba) biti skrivena od ostatka sistema
 - Smanjuje se mogućnost da dođe do greške
- Korisnicima je jasno definisano šta je moguće raditi sa nekim tipom
 - Kako se sprovodi implementacija skriveno je od njega
- Analogija sa kućnim aparatima
 - Korisnici vidi samo skup dugmadi putem kojih koriste aparat
 - Elektronika i mehanika koja vrši operacije su skriveni od korisnika
- Ovaj princip se ostvaruje tako što pojedini dijelovi koda (podaci, funkcije, procedure) mogu označiti kao javni ili privatni



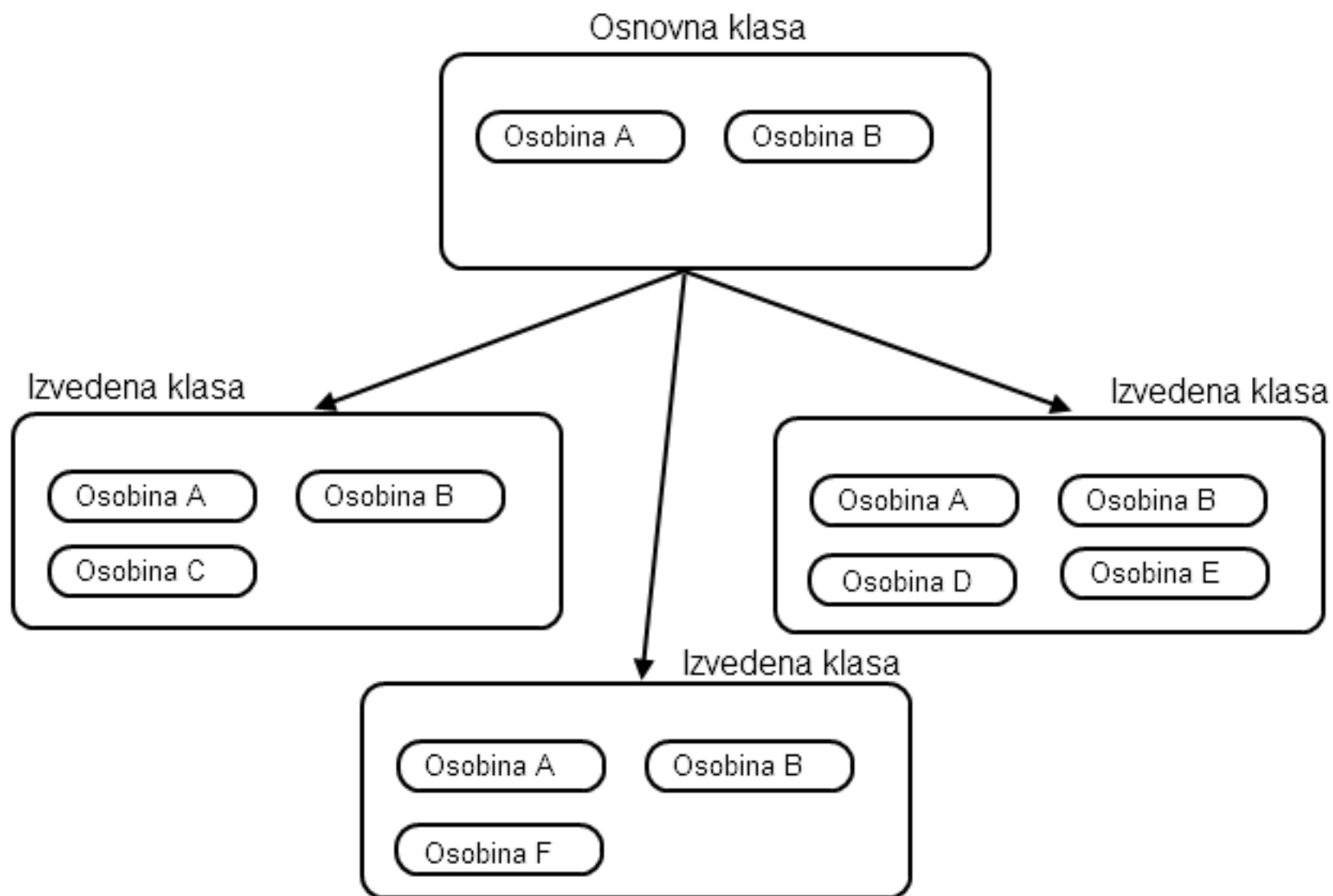
Slika 4. Enkapsulacija

1.3 Temeljni principi – Nasljeđivanje

- Nasljeđivanje
 - Vjerovatno i najvažniji princip OO programiranja
 - U skupu sličnih entiteta mogu se uočiti podskupovi koji dijele neke zajedničke osobine koje razlikuju od ostalih objekata
 - Npr. u skupu entiteta Vozilo mogu se uočiti podskupovi Automobil i Kamion
 - Oni dijele zajedničke osobine jer pripadaju skupu entiteta Vozilo
 - Pored toga, imaju svoje specifične osobine
 - » Za svaki kamion mora biti poznata maksimalna dozvoljena težina tereta

1.3 Temeljni principi – Nasljeđivanje

- Nasljeđivanje
 - Na sličan način klasa A može se izvesti iz klase B
 - Ona zadržava sve funkcionalnosti osnovne klase, tako da se te funkcionalnosti ne moraju ponovo implementirati u izvedenoj klasi
 - Moguće je dodavati nove funkcionalnosti, kao i mijenjati postojeće
 - Ovime se postiže visok stepen ponovne upotrebe programskog koda
 - Jednom kreirana klasa može biti proslijeđena drugim programerima da je koriste
 - » Ako treba unijeti izmjenu programer kreira novu klasu koja se izvodi iz postojeće klase i modifikuje samo određene funkcionalnosti
 - » Programski kod osnovne klase ostaje neizmijenjen
 - » Programer čak ne mora da zna ni jedan detalj implementacije iz osnovne klase
 - Programiranje GUI-a kod većine OO jezika je zasnovana na ovom principu



Slika 5. Nasljeđivanje

1.3 Temeljni principi – Preklapanje operatora i polimorfizam

- Preklapanje operatora
 - Pored mogućnosti uvođenja novih tipova moguće je definisati i značenje operatora za taj novi tip
 - Npr. ako je definisan novi tip `Complex` koji odgovara kompleksnim brojevima, moguće je i definisati značenje operatora `+` nad tim novim tipom
 - Operator `+` u različitim kontekstima ima drugačiji smisao
- Polimorfizam
 - Ovaj pojam vezan je nasljeđivanje
 - Neka u klasi `A` postoji funkcija `Print`
 - Moguće je izvesti klase `B` i `C` iz klase `A` i u tim klasama napisati novu (specifičnu) implementaciju za `Print`
 - Funkcija `Print` ima više oblika
 - » Koji oblik će biti pozvan zavisi od tipa objekta